

# MI Machine Learning

(Makai Marcell - D53G1U)

## Tartalomjegyzék

|          |                                                     |           |
|----------|-----------------------------------------------------|-----------|
| <b>1</b> | <b>BEVEZETÉS.....</b>                               | <b>3</b>  |
| <b>2</b> | <b>GENERÁLT KLASSZIFIKÁCIÓS FELADAT .....</b>       | <b>3</b>  |
| 2.1      | ADATHALMAZ ÉS PARAMÉTEREK .....                     | 3         |
| 2.2      | ALKALMAZOTT OSZTÁLYOZÓK.....                        | 4         |
| 2.3      | PSZEUDOKÓD ÉS UML ACTIVITY DIAGRAM .....            | 4         |
| 2.4      | HIPERPARAMÉTER-HANGOLÁS.....                        | 7         |
| 2.5      | EREDMÉNYEK ÉRTÉKELÉSE .....                         | 7         |
| <b>3</b> | <b>LEARNING RATE ÉS EPOCH VIZSGÁLAT .....</b>       | <b>8</b>  |
| 3.1      | A VIZSGÁLAT CÉLJA .....                             | 8         |
| 3.2      | PSZEUDOKÓD ÉS UML ACTIVITY DIAGRAM .....            | 8         |
| 3.3      | EREDMÉNYEK ÉRTELMEZÉSE .....                        | 10        |
| <b>4</b> | <b>VALÓS ADATHALMAZOS REGRESSZIÓS FELADAT .....</b> | <b>15</b> |
| 4.1      | AIRFOIL SELF-NOISE ADATHALMAZ .....                 | 15        |
| 4.2      | ALKALMAZOTT REGRESSZIÓS MODELLEK.....               | 16        |
| 4.3      | PSZEUDOKÓD ÉS UML ACTIVITY DIAGRAM .....            | 17        |
| 4.4      | HIPERPARAMÉTER-HANGOLÁS.....                        | 19        |
| 4.5      | EREDMÉNYEK ÉRTÉKELÉSE .....                         | 19        |
| <b>5</b> | <b>PROGRAM FELÉPÍTÉSE ÉS FUTTATÁSA .....</b>        | <b>20</b> |
| <b>6</b> | <b>ÖSSZEGZÉS.....</b>                               | <b>21</b> |
| <b>7</b> | <b>IRODALOMJEGYZÉK .....</b>                        | <b>21</b> |

# 1 Bevezetés

A házi feladat célja egyszerű gépi tanulási módszerek alkalmazása klasszifikációs és regressziós problémákon. A megoldás két fő részből áll. Az első részben egy megadott paraméterek alapján generált klasszifikációs adathalmazon három osztályozó algoritmust hasonlítottam össze. A második részben egy valós, közismert adathalmazon regressziós feladatot oldottam meg.

A megoldás Python nyelven, IPython Notebook formátumban készült. A notebook tartalmazza az adathalmazok létrehozását és betöltését, az adatok vizualizációját, a modellek tanítását, a hiperparaméter-hangolást, valamint az eredmények táblázatos és grafikus bemutatását.

Az első részben a Naive Bayes, a Random Forest és a K-Nearest Neighbors osztályozókat vizsgáltam. A második részben az Airfoil Self-Noise adathalmazt használtam [1], ahol a cél a skálázott hangnyomásszint becslése volt regressziós modellek segítségével.

A gépi tanulási modellek megvalósításához a scikit-learn könyvtárat használtam [2], [6]. A numerikus számításokat a NumPy [3], az adatok kezelését a pandas [4], az ábrázolást pedig a matplotlib [5] segítette.

A reprodukálhatóság érdekében a programban rögzített random seed értéket használtam. Ez biztosítja, hogy az adathalmaz generálása, a tanító-teszt bontás és a véletlenszerűséget tartalmazó modellek futtatása ismételhető eredményeket adjon.

## 2 Generált klasszifikációs feladat

### 2.1 Adathalmaz és paraméterek

Az első részfeladatban egy mesterségesen generált klasszifikációs adathalmazt használtam. Az adathalmazt a *scikit-learn make\_classification* függvényével állítottam elő a hozzám tartozó paraméterek alapján.

A generált adathalmaz paraméterei:

|                      |                       |
|----------------------|-----------------------|
| dataset_type         | <i>classification</i> |
| n_samples            | 240                   |
| n_features           | 10                    |
| n_informative        | 5                     |
| n_redundant          | 2                     |
| n_classes            | 2                     |
| n_clusters_per_class | 1                     |
| flip_y               | 0.01                  |
| class_sep            | 1.0                   |
| seed                 | 700664640             |

Az adathalmaz 240 mintát és 10 bemeneti jellemzőt tartalmaz. A 10 jellemzőből 5 informatív, vagyis közvetlenül segíti az osztályok elkülönítését, 2 pedig redundáns, tehát más jellemzőkből származtatott információt hordoz. Az osztályok száma 2, ezért a feladat bináris klasszifikációs problémának tekinthető.

A *flip\_y = 0.01* paraméter azt jelenti, hogy az osztálycímkék kis része véletlenszerűen megváltozhat, így az adathalmaz tartalmazhat kisebb zajt. A *class\_sep = 1.0* az osztályok elkülönülésének mértékét szabályozza.

Mivel az adathalmaz 10 dimenziós, közvetlenül nem ábrázolható teljes egészében kétdimenziós ábrán. Ezért a vizualizációhoz az első két jellemzőt használtam, így a pontdiagram az adatok kétdimenziós vetületét mutatja.

## 2.2 Alkalmazott osztályozók

A generált klasszifikációs adathalmazon három osztályozó algoritmust hasonlítottam össze:

- Naive Bayes
- Random Forest Classifier
- K-Nearest Neighbors Classifier

A **Naive Bayes** egy valószínűségi osztályozó algoritmus, amely Bayes-tételen alapul. Feltételezi, hogy a bemeneti jellemzők egymástól feltételesen függetlenek. Egyszerűsége ellenére sok esetben gyors és hatékony módszer.

A **Random Forest** több döntési fából álló együttes módszer. Minden fa az adatok egy részhalmazán tanul, majd az osztályozás az egyes fák szavazatai alapján történik. Előnye, hogy jól kezeli a nemlineáris összefüggéseket, és kevésbé hajlamos a túltanulásra, mint egyetlen döntési fa.

A **K-Nearest Neighbors** egy példányalapú módszer. Az új mintát az alapján sorolja be, hogy a tanító adatok közül a hozzá legközelebbi k darab szomszéd milyen osztályba tartozik. A módszer érzékeny a jellemzők skálájára, ezért a notebookban *StandardScaler* előfeldolgozást használtam.

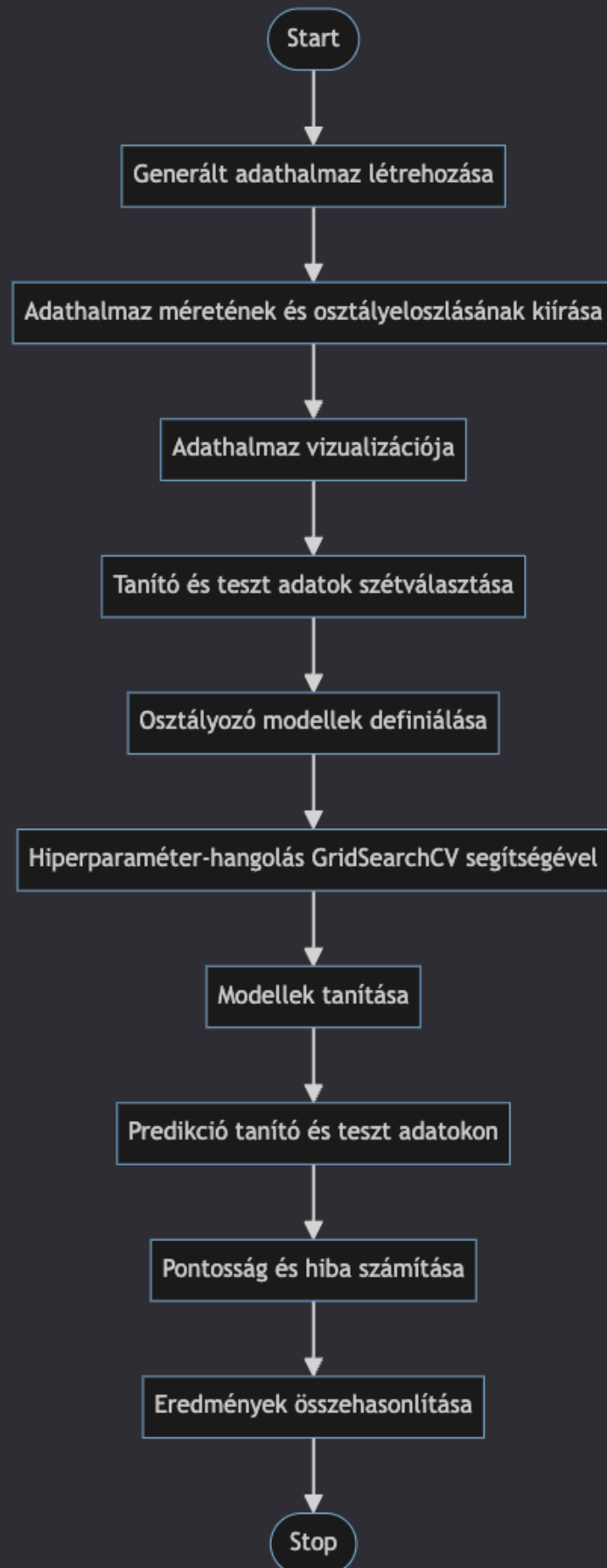
## 2.3 Pszeudokód és UML activity diagram

A klasszifikációs rész általános működése a következő pszeudokóddal írható le:

Bemenet: generált klasszifikációs paraméterek

Kimenet: modellek teljesítményének összehasonlítása

1. Generáljuk le az adathalmazt a megadott paraméterekkel.
2. Írjuk ki az adathalmaz méretét és az osztályeloszlást.
3. Ábrázoljuk az adathalmazt az első két jellemző alapján.
4. Osszuk fel az adatokat tanító és teszt halmazra.
5. Definiáljuk a három osztályozó modellt:
  - Naive Bayes
  - Random Forest
  - K-Nearest Neighbors
6. Minden modell esetén:
  - 6.1. Végezzük el a hiperparaméter-hangolást, ha van hangolandó paraméter.
  - 6.2. Tanítsuk be a modellt a tanító adatokon.
  - 6.3. Készítsünk predikciót a tanító és teszt adatokon.
  - 6.4. Számítsuk ki a pontosságot.
  - 6.5. Számítsuk ki a tanítási és tesztelési hibát.
7. Hasonlítsuk össze az eredményeket táblázatos formában.
8. Ábrázoljuk a tanulási görbéket.



## 2.4 Hiperparaméter-hangolás

A *klasszifikációs* modellek esetén a hiperparaméter-hangolást *GridSearchCV* segítségével végeztem. A *GridSearchCV* több előre megadott hiperparaméter-kombinációt próbál ki, majd keresztvalidáció alapján kiválasztja a legjobb beállítást.

A *Naive Bayes* modellnél nem volt szükség jelentősebb hiperparaméter-hangolásra. A *Random Forest* esetén például a fák számát és a maximális mélységet lehet vizsgálni. A *K-Nearest Neighbors* esetén fontos hiperparaméter a szomszédok száma, valamint az, hogy a szomszédok egyforma vagy távolságalapú súlyt kapjanak.

A klasszifikációs részben a modell teljesítményét elsősorban pontosság alapján értékeltem. A tanítási és tesztelési hiba az alábbi módon számítható:

*tanítási hiba* =  $1 - \text{tanítási pontosság}$

*tesztelési hiba* =  $1 - \text{teszt pontosság}$

## 2.5 Eredmények értékelése

A három osztályozó algoritmus teljesítményét a tanító és teszt adathalmazon elért pontosság, valamint az ezekből számított tanítási és tesztelési hiba alapján hasonlítottam össze. A tanítási hiba és a tesztelési hiba értéke az  $1 - \text{pontosság}$  képlettel számítható. Az eredmények alapján megfigyelhető, hogy a modellek mennyire képesek megtanulni a tanító adatok mintázatait, illetve mennyire jól általánosítanak korábban nem látott teszt adatokra.

Ha egy modell tanítási pontossága magas, de a teszt pontossága jelentősen alacsonyabb, akkor túltanulás gyanítható. Ha mind a tanítási, mind a teszt pontosság alacsony, akkor a modell valószínűleg alultanul. Jó modell esetén a tanítási és tesztelési teljesítmény egyaránt magas, és a két érték között nincs túl nagy eltérés.

A tanulási görbék segítségével megfigyelhető, hogyan változik a tanítási és validációs teljesítmény a tanító adatok mennyiségének függvényében. Ez segít megérteni, hogy a modellnek több adatra lenne-e szüksége, vagy a kiválasztott modell bonyolultsága nem megfelelő.

| Algoritmus          | Tanítási pontosság | Teszt pontosság | Tanítási hiba | Teszt hiba |
|---------------------|--------------------|-----------------|---------------|------------|
| Naive Bayes         | 0.927778           | 0.916667        | 0.072222      | 0.083333   |
| Random Forest       | 1.000000           | 0.933333        | 0.000000      | 0.066667   |
| K-Nearest Neighbors | 0.972222           | 0.916667        | 0.027778      | 0.083333   |

A táblázat alapján a *Random Forest Classifier* érte el a legjobb teszt pontosságot. A modell tanítási pontossága  $1.0$  lett, vagyis a tanító adathalmazon hibátlanul teljesített, míg a teszt adathalmazon is magas pontosságot ért el. Ez arra utal, hogy a modell jól

megtanulta az adathalmaz mintázatait, azonban a tanítási és teszt pontosság közötti különbség miatt enyhe túltanulás lehetősége is felmerülhet.

A *Naive Bayes* és a *K-Nearest Neighbors Classifier* szintén jó eredményt adott, de a teszt pontosságuk valamivel alacsonyabb volt. A három modell közül összességében a *Random Forest Classifier* bizonyult a legerősebb osztályozónak ezen a generált adathalmazon.

## 3 Learning rate és epoch vizsgálat

### 3.1 A vizsgálat célja

A feladatkiírás külön példaként említi a tanulási ráta és az epochok számának vizsgálatát. A három fő klasszifikációs algoritmus közül a *Naive Bayes*, a *Random Forest* és a *K-Nearest Neighbors* nem klasszikus learning rate és epoch alapú tanulási folyamatot használ. Emiatt ezeknél a modelleknél ezek a hiperparaméterek nem értelmezhetők közvetlenül.

A követelmény bemutatása érdekében egy kiegészítő vizsgálatot végeztem *SGDClassifier* modell segítségével. Az *SGDClassifier* gradiens alapú tanulási módszert használ, ezért alkalmas a tanulási ráta és az iterációk, vagyis epochok számának vizsgálatára.

A vizsgált learning rate értékek: [0.001, 0.005, 0.01, 0.05, 0.1, 0.5, 1, 5, 10, 20]

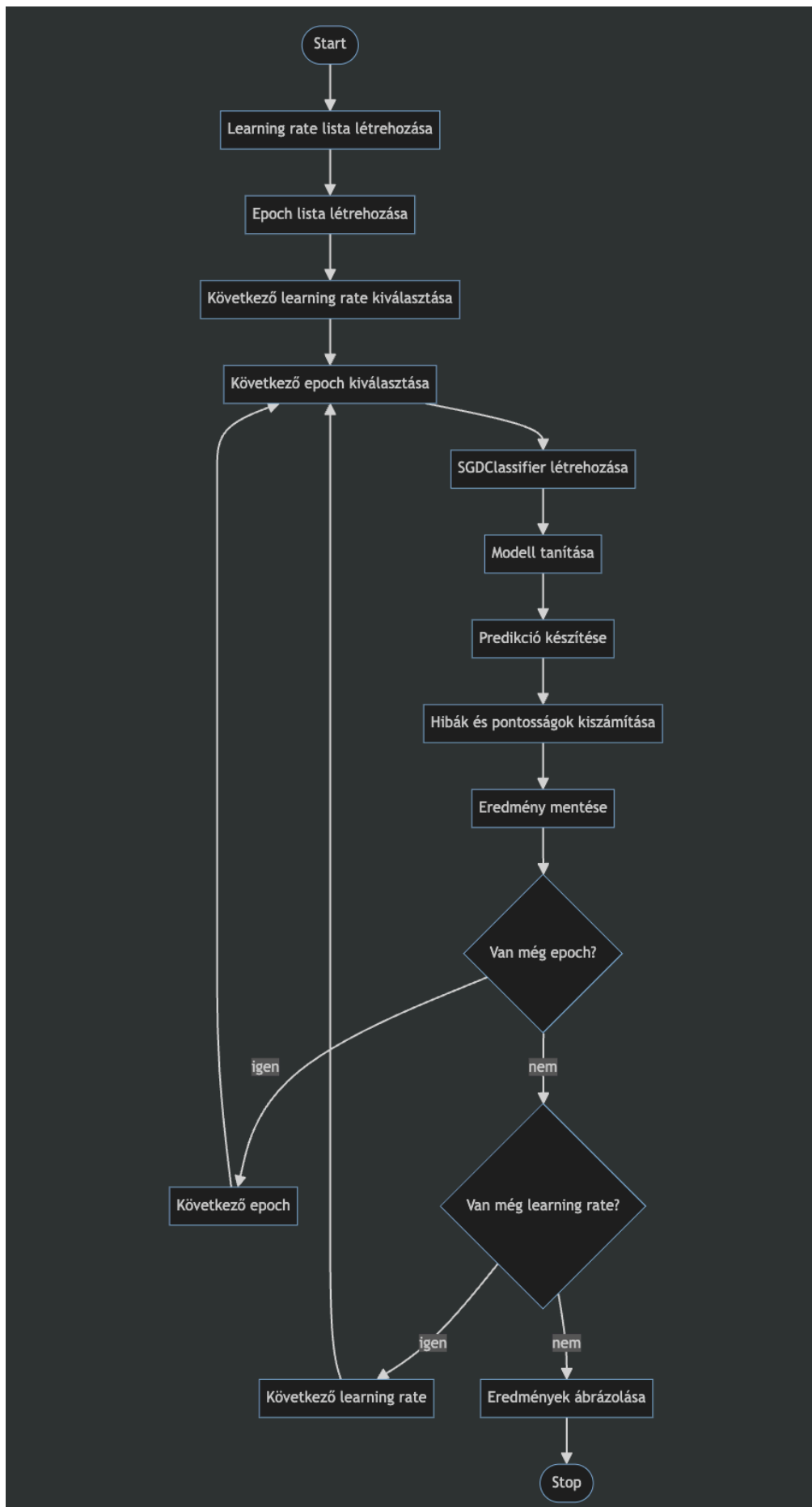
A vizsgált epoch értékek: [25, 50, 100, 500, 1000]

### 3.2 Pszeudokód és UML activity diagram

Bemenet: tanító és teszt adatok, learning rate lista, epoch lista  
Kimenet: learning rate és epoch kombinációk eredményei

1. Hozzuk létre a learning rate értékek listáját.
2. Hozzuk létre az epoch értékek listáját.
3. Minden learning rate értékre:
  - 3.1. Minden epoch értékre:
    - 3.1.1. Hozzunk létre egy *SGDClassifier* modellt.
    - 3.1.2. Állítsuk be az aktuális learning rate és epoch értéket.
    - 3.1.3. Tanítsuk be a modellt a tanító adatokon.
    - 3.1.4. Készítsünk predikciót a tanító és teszt adatokon.
    - 3.1.5. Számítsuk ki a tanítási és teszt pontosságot.
    - 3.1.6. Számítsuk ki a tanítási és teszt hibát.
    - 3.1.7. Mentsük el az eredményt.
4. Rendezzük és jelenítsük meg az eredményeket.
5. Ábrázoljuk a learning rate és epoch hatását.





### 3.3 Eredmények értelmezése

A modell minden learning rate és epoch kombinációval betanításra került. Az eredményeket tanítási pontosság, teszt pontosság, tanítási hiba és teszt hiba alapján értékeltem.

| Learning rate | Epoch | Tanítási pontosság | Teszt pontosság | Tanítási hiba | Teszt hiba | Megjegyzés |               |
|---------------|-------|--------------------|-----------------|---------------|------------|------------|---------------|
| 0             | 0.001 | 25                 | 0.944444        | 0.950000      | 0.055556   | 0.050000   | Sikeres futás |
| 1             | 0.001 | 50                 | 0.955556        | 0.950000      | 0.044444   | 0.050000   | Sikeres futás |
| 2             | 0.001 | 100                | 0.977778        | 0.950000      | 0.022222   | 0.050000   | Sikeres futás |
| 3             | 0.001 | 500                | 0.977778        | 0.950000      | 0.022222   | 0.050000   | Sikeres futás |
| 4             | 0.001 | 1000               | 0.983333        | 0.950000      | 0.016667   | 0.050000   | Sikeres futás |
| 5             | 0.005 | 25                 | 0.977778        | 0.950000      | 0.022222   | 0.050000   | Sikeres futás |
| 6             | 0.005 | 50                 | 0.977778        | 0.950000      | 0.022222   | 0.050000   | Sikeres futás |
| 7             | 0.005 | 100                | 0.977778        | 0.950000      | 0.022222   | 0.050000   | Sikeres futás |
| 8             | 0.005 | 500                | 0.988889        | 0.966667      | 0.011111   | 0.033333   | Sikeres futás |

| Learning rate | Epoch | Tanítási pontosság | Teszt pontosság | Tanítási hiba | Teszt hiba | Megjegyzés |               |
|---------------|-------|--------------------|-----------------|---------------|------------|------------|---------------|
| 9             | 0.005 | 1000               | 0.983333        | 0.966667      | 0.016667   | 0.033333   | Sikeres futás |
| 10            | 0.010 | 25                 | 0.977778        | 0.950000      | 0.022222   | 0.050000   | Sikeres futás |
| 11            | 0.010 | 50                 | 0.983333        | 0.950000      | 0.016667   | 0.050000   | Sikeres futás |
| 12            | 0.010 | 100                | 0.983333        | 0.950000      | 0.016667   | 0.050000   | Sikeres futás |
| 13            | 0.010 | 500                | 0.983333        | 0.966667      | 0.016667   | 0.033333   | Sikeres futás |
| 14            | 0.010 | 1000               | 0.983333        | 0.966667      | 0.016667   | 0.033333   | Sikeres futás |
| 15            | 0.050 | 25                 | 0.983333        | 0.950000      | 0.016667   | 0.050000   | Sikeres futás |
| 16            | 0.050 | 50                 | 0.988889        | 0.966667      | 0.011111   | 0.033333   | Sikeres futás |
| 17            | 0.050 | 100                | 0.983333        | 0.966667      | 0.016667   | 0.033333   | Sikeres futás |
| 18            | 0.050 | 500                | 0.983333        | 0.966667      | 0.016667   | 0.033333   | Sikeres futás |
| 19            | 0.050 | 1000               | 0.983333        | 0.966667      | 0.016667   | 0.033333   | Sikeres futás |

| Learning rate | Epoch | Tanítási pontosság | Teszt pontosság | Tanítási hiba | Teszt hiba | Megjegyzés |               |
|---------------|-------|--------------------|-----------------|---------------|------------|------------|---------------|
| 20            | 0.100 | 25                 | 0.988889        | 0.966667      | 0.011111   | 0.033333   | Sikeres futás |
| 21            | 0.100 | 50                 | 0.983333        | 0.966667      | 0.016667   | 0.033333   | Sikeres futás |
| 22            | 0.100 | 100                | 0.983333        | 0.966667      | 0.016667   | 0.033333   | Sikeres futás |
| 23            | 0.100 | 500                | 0.983333        | 0.966667      | 0.016667   | 0.033333   | Sikeres futás |
| 24            | 0.100 | 1000               | 0.983333        | 0.966667      | 0.016667   | 0.033333   | Sikeres futás |
| 25            | 0.500 | 25                 | 0.977778        | 0.966667      | 0.022222   | 0.033333   | Sikeres futás |
| 26            | 0.500 | 50                 | 0.944444        | 0.983333      | 0.055556   | 0.016667   | Sikeres futás |
| 27            | 0.500 | 100                | 0.983333        | 0.966667      | 0.016667   | 0.033333   | Sikeres futás |
| 28            | 0.500 | 500                | 0.983333        | 0.966667      | 0.016667   | 0.033333   | Sikeres futás |
| 29            | 0.500 | 1000               | 0.977778        | 0.983333      | 0.022222   | 0.016667   | Sikeres futás |
| 30            | 1.000 | 25                 | 0.977778        | 0.966667      | 0.022222   | 0.033333   | Sikeres futás |

| Learning rate | Epoch  | Tanítási pontosság | Teszt pontosság | Tanítási hiba | Teszt hiba | Megjegyzés |               |
|---------------|--------|--------------------|-----------------|---------------|------------|------------|---------------|
| 31            | 1.000  | 50                 | 0.894444        | 0.950000      | 0.105556   | 0.050000   | Sikeres futás |
| 32            | 1.000  | 100                | 0.983333        | 0.983333      | 0.016667   | 0.016667   | Sikeres futás |
| 33            | 1.000  | 500                | 0.977778        | 0.966667      | 0.022222   | 0.033333   | Sikeres futás |
| 34            | 1.000  | 1000               | 0.972222        | 0.983333      | 0.027778   | 0.016667   | Sikeres futás |
| 35            | 5.000  | 25                 | 0.966667        | 0.966667      | 0.033333   | 0.033333   | Sikeres futás |
| 36            | 5.000  | 50                 | 0.927778        | 0.950000      | 0.072222   | 0.050000   | Sikeres futás |
| 37            | 5.000  | 100                | 0.955556        | 0.933333      | 0.044444   | 0.066667   | Sikeres futás |
| 38            | 5.000  | 500                | 0.961111        | 0.950000      | 0.038889   | 0.050000   | Sikeres futás |
| 39            | 5.000  | 1000               | 0.944444        | 0.933333      | 0.055556   | 0.066667   | Sikeres futás |
| 40            | 10.000 | 25                 | 0.966667        | 0.950000      | 0.033333   | 0.050000   | Sikeres futás |
| 41            | 10.000 | 50                 | 0.900000        | 0.933333      | 0.100000   | 0.066667   | Sikeres futás |

| Learning rate | Epoch  | Tanítási pontosság | Teszt pontosság | Tanítási hiba | Teszt hiba | Megjegyzés |               |
|---------------|--------|--------------------|-----------------|---------------|------------|------------|---------------|
| 42            | 10.000 | 100                | 0.894444        | 0.933333      | 0.105556   | 0.066667   | Sikeres futás |
| 43            | 10.000 | 500                | 0.950000        | 0.950000      | 0.050000   | 0.050000   | Sikeres futás |
| 44            | 10.000 | 1000               | 0.950000        | 0.966667      | 0.050000   | 0.033333   | Sikeres futás |
| 45            | 20.000 | 25                 | 0.972222        | 0.966667      | 0.027778   | 0.033333   | Sikeres futás |
| 46            | 20.000 | 50                 | 0.900000        | 0.950000      | 0.100000   | 0.050000   | Sikeres futás |
| 47            | 20.000 | 100                | 0.977778        | 0.966667      | 0.022222   | 0.033333   | Sikeres futás |
| 48            | 20.000 | 500                | 0.961111        | 0.950000      | 0.038889   | 0.050000   | Sikeres futás |
| 49            | 20.000 | 1000               | 0.944444        | 0.950000      | 0.055556   | 0.050000   | Sikeres futás |

A táblázat alapján megfigyelhető, hogy a modell teljesítménye érzékenyen reagál a learning rate értékére. Alacsonyabb learning rate esetén a tanulás stabilabb, de a modellnek több tanítási iterációra lehet szüksége. Nagyobb learning rate esetén a modell gyorsabban változtatja a súlyokat, azonban a túl nagy értékek instabilabb eredményhez és magasabb teszt hibához vezethetnek.

A vizsgált beállítások közül a legjobb eredményt a *learning\_rate = 1.0* és *epoch = 100* kombináció adta. Ennél a beállításnál a tanítási pontosság és a teszt pontosság is 0.983333 lett, a tanítási és teszt hiba pedig egyaránt 0.016667 volt. Ez azt jelenti, hogy a

modell jól általánosított, mert a tanító és teszt adatokon mért teljesítmény között nem volt jelentős különbség.

Néhány nagyobb learning rate értéknél a tesztelési teljesítmény gyengébb lett. Például  $learning\_rate = 20$  és  $epoch = 50$  mellett a teszt pontosság  $0.950000$  volt, miközben a tanítási pontosság csak  $0.900000$  lett. A  $learning\_rate = 10$  értékeknél is ingadozás látható, ami arra utal, hogy a nagy tanulási ráta instabilabb tanítást eredményezhet.

Az epochok számának növelése önmagában nem minden esetben javította a tesztelési eredményt. Például alacsony learning rate mellett a tanítási teljesítmény javulhatott, miközben a teszt pontosság nem változott jelentősen. Ez azt mutatja, hogy több epoch nem feltétlenül jelent jobb általánosítási képességet.

Összességében a vizsgálat alapján a learning rate és az epochok száma fontos hatással volt az *SGDClassifier* teljesítményére. A túl alacsony learning rate lassabb tanulást, a túl magas learning rate pedig instabilabb tanítást eredményezhet. A legjobb beállításnál nem volt jelentős különbség a tanítási és tesztelési hiba között, ezért ennél a konfigurációnál nem volt egyértelmű túltanulás.

## 4 Valós adathalmazos regressziós feladat

### 4.1 Airfoil Self-Noise adathalmaz

A második részfeladatban egy valós, közismert adathalmazt kellett használni úgy, hogy az adatbetöltés ne az `sklearn.datasets` modulból történjen. Ehhez az **Airfoil Self-Noise** adathalmazt használtam, amely az **UCI Machine Learning Repository**ből származik.

Az adathalmaz aerodinamikai és akusztikai méréseket tartalmaz. A feladat regressziós probléma, mivel a célváltozó folytonos numerikus érték. A modell célja a skálázott hangnyomásszint becslése a bemeneti jellemzők alapján.

Az adathalmazt a következő nyers URL-ről töltöttem be: [https://archive.ics.uci.edu/ml/machine-learning-databases/00291/airfoil\\_self\\_noise.dat](https://archive.ics.uci.edu/ml/machine-learning-databases/00291/airfoil_self_noise.dat)

Az adathalmaz 1503 sort és 6 oszlopot tartalmaz. Ezek közül 5 oszlop bemeneti jellemző, 1 oszlop pedig célváltozó.

| Oszlop neve                                  | Típus     | Jelentés                                         |
|----------------------------------------------|-----------|--------------------------------------------------|
| <i>frequency_hz</i>                          | numerikus | Frekvencia                                       |
| <i>angle_of_attack_deg</i>                   | numerikus | Támadási szög                                    |
| <i>chord_length_m</i>                        | numerikus | Húrhossz                                         |
| <i>free_stream_velocity_ms</i>               | numerikus | Szabad áramlási sebesség                         |
| <i>suction_side_displacement_thickness_m</i> | numerikus | Szívóoldali elmozdulási vastagság                |
| <i>scaled_sound_pressure_db</i>              | numerikus | Skálázott hangnyomásszint decibelben, célváltozó |

A célváltozó a *scaled\_sound\_pressure\_db*, amely a zajszintet írja le decibelben. Mivel ez folytonos érték, az adathalmaz regressziós feladatként kezelhető.

A notebookban az adathalmaz beolvasása után ellenőriztem az adathalmaz méretét, az oszlopok adattípusait, valamint a leíró statisztikákat. Emellett ábrázoltam a célváltozó eloszlását is, hogy látható legyen, milyen tartományban helyezkednek el a becslendő értékek.

## 4.2 Alkalmazott regressziós modellek

A regressziós feladat megoldására több modellt hasonlítottam össze. A cél az volt, hogy megvizsgáljam, melyik modell képes legjobban becsülni a skálázott hangnyomásszintet.

Az alkalmazott regressziós modellek:

- Linear Regression
- Ridge Regression
- K-Nearest Neighbors Regressor
- Random Forest Regressor

A **Linear Regression** egyszerű lineáris kapcsolatot feltételez a bemeneti jellemzők és a célváltozó között. Előnye, hogy gyorsan tanítható és könnyen értelmezhető. Hátránya, hogy összetett, nemlineáris kapcsolatok esetén gyengébb eredményt adhat.

A **Ridge Regression** a lineáris regresszió regularizált változata. Az alpha hiperparaméter szabályozza a regularizáció erősségét. A regularizáció célja a túlilleszkedés csökkentése, mivel bünteti a túl nagy modell-együtthatókat.

A **K-Nearest Neighbors Regressor** a legközelebbi szomszédok alapján becsüli meg a célváltozót. Az új minta becsült értéke a hozzá legközelebb eső tanító minták célértékeiből számítható. A modell érzékeny a bemeneti jellemzők skálájára, ezért *StandardScaler* előfeldolgozást használtam.

A **Random Forest Regressor** több döntési fából álló együttes modell. Minden fa külön becslést ad, a végső predikció pedig az egyes fák becsléseinek átlaga. Ez a módszer jól kezeli a nemlineáris összefüggéseket, ezért a valós adathalmazos regressziós feladatban jó teljesítményt nyújthat.

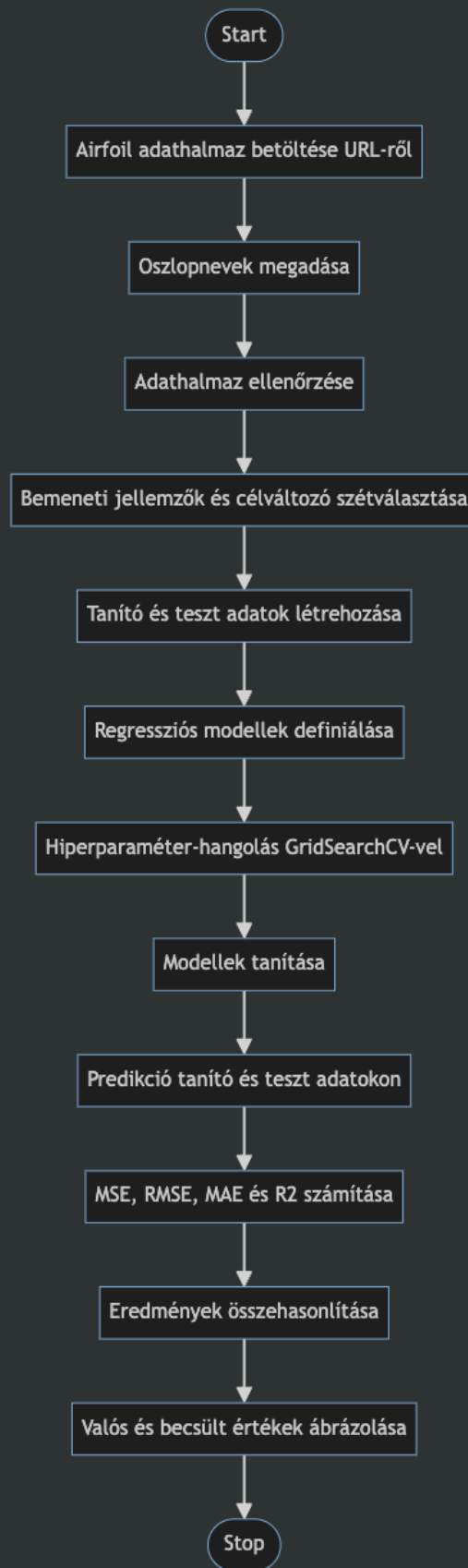


### 4.3 Pszeudokód és UML activity diagram

Bemenet: Airfoil Self-Noise adathalmaz

Kimenet: regressziós modellek kiértékelése

1. Töltsük be az Airfoil Self-Noise adathalmazt külső URL-ről.
2. Adjunk neveket az oszlopoknak.
3. Ellenőrizzük az adathalmaz méretét és adattípusait.
4. Válasszuk szét a bemeneti jellemzőket és a célváltozót.
5. Osszuk fel az adatokat tanító és teszt halmazra.
6. Definiáljuk a regressziós modelleket.
7. Minden regressziós modell esetén:
  - 7.1 Ha szükséges, végezzünk skálázást.
  - 7.2 Ha vannak hiperparaméterek, végezzünk GridSearchCV hangolást.
  - 7.3 Tanítsuk be a modellt.
  - 7.4 Készítsünk predikciót a tanító és teszt adatokon.
  - 7.5 Számítsuk ki az MSE, RMSE, MAE és R2 értékeket.
  - 7.6 Mentsük el az eredményeket.
8. Hasonlítsuk össze a modelleket táblázatos formában.
9. Ábrázoljuk a valós és becsült értékeket.



## 4.4 Hiperparaméter-hangolás

A regressziós modellek hiperparaméter-hangolását *GridSearchCV* segítségével végeztem. A *GridSearchCV* több megadott paraméterkombinációt próbál ki, majd keresztvalidáció alapján kiválasztja a legjobb beállítást.

A *Linear Regression* modellnél nem használtam külön hiperparaméter-hangolást, mivel az alapmodellnek ebben a feladatban nem volt lényeges hangolandó hiperparamétere.

A *Ridge Regression* esetén az alpha regularizációs paramétert vizsgáltam. Az alpha értéke meghatározza, hogy a modell mennyire bünteti a nagy együttthatókat. Kis alpha esetén a modell közelebb áll az egyszerű lineáris regresszióhoz, nagyobb alpha esetén erősebb a regularizáció.

A *K-Nearest Neighbors Regressor* esetén a szomszédok számát és a súlyozás módját hangoltam. A *n\_neighbors* azt határozza meg, hogy hány közeli minta alapján történjen a becslés. A *weights* paraméter esetén az uniform minden szomszédot azonos súllyal vesz figyelembe, míg a *distance* súlyozás a közelebbi szomszédokat nagyobb súllyal kezeli.

A *Random Forest Regressor* esetén a fák számát, a maximális mélységet és a csomópontfelosztáshoz szükséges minimális mintaszámot vizsgáltam. A fák száma hatással lehet a modell stabilitására, a maximális mélység pedig a modell bonyolultságát befolyásolja.

A lineáris és KNN-alapú modelleknél *StandardScaler* előfeldolgozást alkalmaztam. Erre azért volt szükség, mert ezek a modellek érzékenyek lehetnek a jellemzők eltérő nagyságrendjére. A *Random Forest* esetén a skálázás nem szükséges, mert a döntési fákon alapuló modellek kevésbé érzékenyek a bemeneti változók skálájára.

A regressziós modelleket a *neg\_mean\_squared\_error* metrika alapján hangoltam. Ennek oka, hogy a scikit-learn a nagyobb értékeket tekinti jobbnak, ezért a négyzetes hiba negatív előjellel szerepel az optimalizálás során.

## 4.5 Eredmények értékelése

A regressziós modellek teljesítményét több metrikával értékeltem:

|             |                           |
|-------------|---------------------------|
| <b>MSE</b>  | átlagos négyzetes hiba    |
| <b>RMSE</b> | az MSE négyzetgyöke       |
| <b>MAE</b>  | átlagos abszolút hiba     |
| <b>R2</b>   | determinációs együtttható |

Az **MSE** a valós és a becsült értékek különbségének négyzetes átlaga. Minél kisebb az **MSE**, annál pontosabb a modell. A négyzetre emelés miatt a nagyobb hibák erősebben büntetésre kerülnek.

Az **RMSE** az **MSE** négyzetgyöke, ezért az eredeti célváltozóval azonos mértékegységben értelmezhető. A **MAE** az abszolút hibák átlaga, amely egyszerűen megmutatja, hogy a modell átlagosan mekkorát téved. Az **R2** azt mutatja meg, hogy a modell a célváltozó varianciájának mekkora részét képes megmagyarázni.

| Modell                        | Teszt MSE | Teszt RMSE | Teszt MAE | Teszt R2 |
|-------------------------------|-----------|------------|-----------|----------|
| Random Forest Regressor       | 3.745828  | 1.935414   | 1.400188  | 0.925422 |
| K-Nearest Neighbors Regressor | 6.402694  | 2.530354   | 1.846736  | 0.872524 |
| Linear Regression             | 26.383853 | 5.136521   | 3.988916  | 0.474706 |
| Ridge Regression              | 26.471693 | 5.145065   | 4.005158  | 0.472957 |

A táblázat alapján a *Random Forest Regressor* érte el a legjobb eredményt. A tesztalmazon mért *MSE* értéke 3.7458, az *RMSE* értéke 1.9354, az *MAE* értéke 1.4002, az *R2* értéke pedig 0.9254 lett. Ez azt jelenti, hogy a modell a célváltozó varianciájának több mint **92%**-át képes volt megmagyarázni.

A *K-Nearest Neighbors Regressor* szintén jó eredményt adott, azonban a tanító adatokon lényegesen jobb eredményt ért el, mint a teszt adatokon, ami enyhe túltanulásra utalhat. A *Linear Regression* és *Ridge Regression* gyengébben teljesített, ami arra utal, hogy az *Airfoil Self-Noise* adathalmaz összefüggései nem írhatók le megfelelően egyszerű lineáris modellekkel.

A regressziós modell működésének szemléltetésére a valós és a becsült célértékeket is ábrázoltam. Az ábrán minden pont egy tesztmintát jelöl. Ideális esetben a pontok az átlós referenciaegyenes közelében helyezkednek el, mert ez azt jelentené, hogy a becsült értékek megegyeznek a valós értékekkel.

Az ábra alapján látható, hogy a *Random Forest Regressor* becslései nagyrészt követik a valós értékeket. A pontok jelentős része az ideális egyenes közelében helyezkedik el, ami összhangban van a modell alacsony *MSE* és magas *R2* értékével. Ez vizuálisan is megerősíti, hogy a modell jó becslést adott az *Airfoil Self-Noise* adathalmazon.

## 5 Program felépítése és futtatása

A notebook cellákra bontva tartalmazza az importokat, az adathalmazok létrehozását és betöltését, a modellek tanítását, a hiperparaméter-hangolást, az eredmények kiértékelését és a vizualizációkat.

A notebook futtatása parancssorból: „*jupyter notebook D53G1U.ipynb*”

A szükséges csomagok telepítése: „*pip install numpy pandas matplotlib scikit-learn notebook*”

## 6 Összegzés

A házi feladatban sikerült megvalósítani egy generált klasszifikációs és egy valós adathalmazos regressziós gépi tanulási feladatot. Az első részben a D53G1U Neptun-kódhoz tartozó paraméterek alapján generált adathalmazon három osztályozó algoritmust hasonlítottam össze: Naive Bayes, Random Forest és K-Nearest Neighbors.

A modellek teljesítményét pontosság, tanítási hiba és tesztelési hiba alapján értékeltem. A hiperparaméter-hangoláshoz GridSearchCV módszert használtam. Kiegészítő vizsgálatként SGDClassifier segítségével elemeztem a tanulási ráta és az epochok számának hatását is.

A második részben az Airfoil Self-Noise adathalmazt használtam regressziós feladat megoldására. Az adathalmazt külső URL-ről töltöttem be, nem az sklearn.datasets modulból. A regressziós modellek teljesítményét MSE, RMSE, MAE és R2 metrikák alapján hasonlítottam össze.

A regressziós részben a Random Forest Regressor érte el a legjobb eredményt. A modell tesztalmazon mért R2 értéke meghaladta a 0,90-es értéket, így a modell teljesítménye elérte a feladatban megfogalmazott körülbelül 90%-os célkitűzést. Az eredmények alapján a döntési fákból álló együttes modell jobban kezelte az adathalmaz nemlineáris összefüggéseit, mint az egyszerű lineáris regressziós modellek.

A feladat megoldása során jól láthatóvá vált, hogy a különböző gépi tanulási algoritmusok eltérő módon viselkednek ugyanazon adathalmazon. A modellválasztás, az előfeldolgozás és a hiperparaméter-hangolás jelentősen befolyásolja a végső teljesítményt.

## 7 Irodalomjegyzék

[1] T. Brooks, D. Pope, and M. Marcolini, “Airfoil Self-Noise,” UCI Machine Learning Repository, 1989. doi: 10.24432/C5VW2C.

[2] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[3] NumPy Developers, “NumPy documentation.” [Online]. Available: <https://numpy.org/doc/>

[4] pandas Developers, “pandas documentation.” [Online]. Available: <https://pandas.pydata.org/docs/>

[5] Matplotlib Developers, “Matplotlib documentation.” [Online]. Available: <https://matplotlib.org/stable/>

[6] scikit-learn Developers, “scikit-learn User Guide.” [Online]. Available: [https://scikit-learn.org/stable/user\\_guide.html](https://scikit-learn.org/stable/user_guide.html)